

Table of Contents

Zombie Survivors	1
Rapport 1	1
Rapport 2 (prototype)	3
Rapport 3 (early access)	8
Rapport 4	10
Eksamensoppgaver	13
1.1 Bakgrunn [5 poeng]	13
1.2 Teamarbeid [5 poeng]	14
1.3 Fremtid [5 poeng]	15
2.1 Git arbeidsflyt [6 poeng]	15
2.2 IDE [4 poeng]	17
2.3 CI/CD [3 poeng]	18
2.4 Statisk analyse [2 poeng]	19
3.1 Valg av abstraksjoner [5 poeng]	21
3.2 Å mutere eller ikke mutere? [2 poeng]	22
3.3 Java interface [6 poeng]	22

Zombie Survivors

Rapport 1

Team: Out of bounds - Martin B, Felix, Sebastian, Nickolay, Daniel, Martin SF

Team / project

Roles

- Nickolay - Project manager
- Sebastian - GitLab administrator
- Daniel - Kanban lead
- Martin F - Manual testing lead
- Felix - Automated testing lead
- Martin B - UX lead

The roles are preliminary and may be adjusted as the project develops. At this point the main responsibilities are: Nickolay organizes meetings and keeps the overview, Sebastian manages the GitLab repository, Daniel will set up and maintain the Kanban board, Martin F will coordinate manual playtesting, Felix will own automated tests and CI, and Martin B will handle design and UX decisions.

There is no formal customer contact role since the project is self-directed. Nickolay handles contact with the course staff.

Project methodology

We plan to use Kanban with a board to track tasks. Meetings are scheduled for Thursdays and Fridays. Communication between meetings happens on Discord. We have not yet finalized all process details and will adjust as we learn what works.

Group dynamics and communication

Everyone participated in the first meeting. All team members have cloned the repo and gotten the project running. Communication has been active and there are no issues so far.

Retrospective

What we accomplished:

- Set up the team and repository
- Defined roles
- Wrote a specification with concept description, MVP requirements, and user stories
- Removed starter project code and set up our own project identity (name, README, license)
- Everyone got the project running locally

What can be improved:

- We have not started coding gameplay yet
- The Kanban board is not set up yet
- We need to agree on MVC approach before we start building features

Improvement points for next sprint

1. Set up the Kanban board and start using it for task tracking
2. Start coding core gameplay (player movement, enemies)
3. Agree on an MVC architecture approach before building features

Requirements and specification

We have written a specification document: spec.md

The specification includes a concept description (top-down horde survival game inspired by Vampire Survivors), a list of 10 MVP requirements, and three prioritized user stories.

MVP requirements (not started)

- Show a game board
- Show player on game board
- Move player
- Player interacts with terrain
- Player has XP and can level up
- Show enemies; they interact with terrain and player
- Player can die (from enemy contact)
- Goal: survive as long as possible
- Enemies spawn continuously

- Start screen and game over screen

No changes have been made to the MVP requirements.

User stories

See spec.md for the full list. The three prioritized user stories are:

- As a player I need a character I can move and shoot with so I can play the game.
- As a player I need enemies to fight, such as zombies.
- As a player I need upgrades to get progression in the game.

Acceptance criteria and detailed work tasks will be added for the next submission as we begin implementing features.

Priority

We will focus on MVP first, starting with player movement and enemies. Features beyond MVP will be planned after MVP is reached.

Code

We have set up the project, removed the starter code (obligator.png, blipp.ogg), updated the README, and added a license. The project compiles and runs. We have experimented briefly with LibGDX to understand the framework. No gameplay code has been written yet.

Rapport 2 (prototype)

Report - submission 2 (prototype)

Team: Out of bounds - Martin B, Felix, Sebastian, Nickolay, Daniel, Martin SF

Team / project

Roles:

- Nickolay: Project manager
- Sebastian: GitLab administrator
- Daniel: Kanban lead
- Martin F: Manual testing lead
- Felix: Automated testing lead
- Martin B: UX lead

In practice, roles became more concrete during coding. Felix focused on menus, Martin SF on the map, Martin B on the HUD, Nickolay on animations/sprites, Daniel on

enemies, and Sebastian on git/architecture. Everyone stayed flexible beyond their named roles.

Methodology:

We used a Kanban board to track tasks and met on Thursdays and Fridays. The rhythm worked well.

A major lesson was that MVC was harder to apply in LibGDX than expected. Too much logic initially ended up in the view layer, so we refactored to separate model, view, and controller properly. This was painful but improved the structure.

We also tried A* pathfinding for enemies, but dropped it because it was hard to maintain and performed poorly. Simpler zombie movement fit the theme better.

Group dynamics:

Communication happened mainly on Discord and in person. Everyone contributed actively, and there were no conflicts. Disagreements were resolved through discussion and majority agreement.

Retrospective:**Went well:**

- Playable prototype with player, enemies, shooting, map, and menus
- MVC refactor improved structure
- Good division of work
- Dropping A* was decided quickly

Could improve:

- Enforce MVC separation earlier
- Start testing
- Finish incomplete features such as enemy sprites, map boundaries, and ammo system

Next sprint:

1. Add enemy sprites and map boundaries
2. Start writing tests alongside features
3. Create Kanban issues before starting work

Commit balance:

Commits are reasonably distributed. Everyone committed code, though some used smaller frequent commits while others batched larger changes.

Meeting minutes:

referat-2.md

Requirements and specification

MVP progress:

1. Show game board: Done
2. Show player on game board: Done
3. Move player with WASD: Done
4. Terrain blocks player movement: Done
5. XP and leveling: Not started
6. Enemies interact with terrain/player: Partially done
7. Player can die from enemy contact: Done
8. Goal is to survive as long as possible: Not started
9. Enemies spawn continuously: Partially done
10. Start screen and game over screen: Done

No MVP requirements changed.

User story 1: Player movement and shooting

As a player, I need a character I can move and aim so I can interact with the game world.

Acceptance criteria:

- Move with WASD
- Face mouse cursor
- Shoot toward mouse cursor
- Bullets collide with trees and enemies

Tasks:

- Implement WASD movement
- Implement mouse aiming
- Implement bullet spawning/direction
- Implement bullet/tree and bullet/enemy collision

Status: Done

User story 2: Enemies

As a player, I need enemies to fight so the game has challenge.

Acceptance criteria:

- Enemies spawn and walk toward player
- Enemies damage player on contact
- Enemies die after enough damage

Tasks:

- Implement enemy movement
- Implement spawning
- Implement contact damage and health
- Implement enemy/tree collision

Status: Partially done. Basic movement and damage work, but sprites and map boundaries are missing.

User story 3: Map and terrain

As a player, I need a world to navigate so the game has spatial challenge.

Acceptance criteria:

- Trees block player, enemies, and bullets
- Camera follows player
- World is larger than screen

Tasks:

- Implement tilemap with trees
- Implement collision with trees
- Implement camera follow

Status: Done

Prioritization:

We prioritized MVP requirements first. Movement and shooting came first because other features depend on a controllable player. Enemies came next as the core challenge. Menus/screens were done in parallel. XP and upgrades are deferred until core gameplay is solid.

Known bugs:

- Enemies can get stuck if they spawn outside the intended map area
- Player can walk outside the map boundary

Code

Refactoring:

We refactored to separate MVC properly. The model now handles game state and logic, the view handles rendering, and the controller handles input.

We also added an event system so the view can react to model events such as shots and deaths without direct coupling.

Architecture:

Model: game state and logic, including player, enemies, weapons, and map

View: rendering with LibGDX SpriteBatch, HUD, and menus

Controller: input handling and delegation to model

The model fires events, and the view registers listeners.

Code quality:

The codebase is better structured after refactoring, but collision handling is still scattered and should be consolidated.

Testing:

There are very few tests so far. A fake/empty map setup was created so tests do not depend on real assets. Test coverage should be prioritized next sprint.

Need help with:

MVC in LibGDX is harder than expected. We would benefit from examples of how other LibGDX projects separate model and view, especially around input and animation.

Rapport 3 (early access)**Report - submission 3 (early access)**

Team: Out of bounds - Martin B, Felix, Sebastian, Nickolay, Daniel, Martin SF

Team/project:

Roles are still working fine: Nickolay = project manager, Sebastian = GitLab administrator, Daniel = Kanban lead, Martin F = manual testing lead, Felix = automated testing lead, Martin B = UX lead. Since MVP has been reached, roles are more flexible and everyone contributes across testing and documentation.

Methodology:

Kanban has worked well. We meet Thursdays and Fridays. We should have been better at creating issues and linking branches to them, since some features were merged without corresponding issues.

Group dynamics:

Good. Communication is mainly through Discord and in-person meetings. No unresolved disagreements.

Retrospective:

Went well: good work division, flexible roles, MVC refactoring, solid feature set with enemy types, progression, sound, and HUD.

Can improve: tests started too late, Javadoc/documentation added too late, issues/MRs used inconsistently, sometimes direct commits to main.

Next sprint improvements:

1. Reach 75% test coverage before final submission
2. Write Javadoc for all interfaces and public methods
3. Use issues and merge requests consistently

Commits:

Reasonably distributed. Everyone contributed code. Differences mostly come from commit style.

Meeting minutes:

referat-3.md

Requirements/specification:

All 10 MVP requirements are done and the game is beyond MVP. No MVP requirements have changed.

MVP done:

1. Game board
2. Player on board
3. Player movement
4. Terrain interaction
5. XP and leveling
6. Enemies with terrain/player interaction
7. Player death
8. Survival goal
9. Continuous enemy spawning
10. Start and game-over screens

Beyond MVP:

Police zombie that shoots, level-up upgrades, HUD with score/timer, health/ammo drops, sound/music, blood spots and particles.

User stories done:

US4 Combat feedback: sound, particles, music, and event-based sound separation.

US5 Progression: XP, level thresholds, upgrade screen, upgrades.

US6 Enemy variety: police zombie with shooting behavior and variant spawning.

US7 Resource management: ammo, reload, health/ammo pickups, pickup collision, drop factory.

Priorities:

Soft feature freeze. Main focus is code structure, tests, documentation, cleanup, and only small helpful features.

Known bug:

Some upgrade combinations become too strong too quickly.

Code:

AI-assisted development: Claude was tried for A* pathfinding, but the result was hard to understand, hard to maintain, and performed poorly. We replaced it with a simpler solution ourselves.

Refactoring since submission 2:

Tighter MVC, better encapsulation, event system, factories for enemies/bullets/drops/players, and bug fixes for spawning and map boundaries.

Architecture:

MVC with model, view, controller. Events decouple model and view. Factories make object creation consistent and testable.

Code quality:

Improved encapsulation, package READMEs, tighter MVC, factory pattern. Codebase is readable enough for all members to navigate and modify.

Testing:

Coverage excluding view is not yet 75% for most types. Current coverage is roughly 55-68%, except class coverage at 87.2%.

Coverage:

Line: 1084/1602 = 67.7%

Branch: 391/725 = 53.9%

Instruction: 4626/6743 = 68.6%

Method: 266/393 = 67.7%

Class: 34/39 = 87.2%

Complexity: 399/764 = 52.2%

Needs more tests:

Collision, spawning, drops, upgrades, weapons, and game state transitions.

Going forward:

Focus on test coverage and Javadoc before final submission. No significant external help is needed.

Rapport 4

Report – Submission 4

Team: Out of Bounds – Martin B, Felix, Sebastian, Nickolay, Daniel, Martin SF

Team / Project

Roles

Our roles became more flexible throughout the project. We started with specific areas like tests, git, MVC structure, player textures and collision logic, but eventually focused

more on what was most interesting or needed at the time. As a result, everyone contributed to and understands most of the project.

We also improved at meeting regularly, both in person and on Discord. This made cooperation easier without needing strict roles. Next time, we might define clearer roles from the start while still staying flexible.

Experiences / Project Methodology

Group Dynamics and Communication

The group dynamics worked well. We used Discord for scheduling, questions and discussions, and often met in person to work together. Communication was open, with a low barrier for asking for help or giving feedback.

We kept the code readable and used JavaDocs for important classes and methods. Interfaces and the factory pattern made it easier to add weapons, enemies, drops and other objects without changing large parts of the codebase. Disagreements were mostly solved through discussion, testing and evaluating what worked best.

Retrospective

What Went Well

We are happy with the final result. We made a fun, fully playable game and learned a lot about teamwork and git. Our strongest points were good communication, motivation, and thinking through implementations before writing code. Using factories and events early also paid off.

What Could Be Improved

The biggest issue was scope. Even when we agreed to refactor and polish, we kept adding new features. This was fun, but affected code quality and balancing. Upgrades especially became difficult to tune and often made the player too strong.

Architecturally, AppModel and AppView became too large and had too many responsibilities. Splitting and refactoring them became a recurring task, and this should have been handled more clearly from the start.

What We Did

In this phase, we focused on refactoring and smaller improvements based on playtesting, since this is the release version.

Features: Added new level-up weapons, upgrades for them, and stronger boss enemies.

Tests: Increased focus on testing. Coverage is above our branch coverage goal and well above the instruction coverage requirement.

Encapsulation: Continued using `get()` methods so fields can remain private.

Refactoring: Reduced complexity in `AppModel` and `AppView` by moving responsibilities into smaller components and improving factory pattern usage.

Bug fixes: Fixed a shotgun asset bug where it used some assault rifle sprites.

Code

Architecture and Design Choices

The project uses MVC architecture and event-driven design to keep the model and view decoupled. The model can notify the view about game events without knowing how they are displayed.

Features are split into dedicated classes, such as `PlayerModel` and `EnemyModel`, which build on `LivingObject` and helper classes. The view is also split into separate classes for the HUD, death screen, menus and similar components.

The factory pattern is used for enemies, bullets, drops and players, making object creation cleaner and easier to extend and test. Adding a new enemy or weapon only requires changing a few files.

The main architectural issue is that `AppModel` and `AppView` became too large. `AppModel` still has more dependencies than we would like, but fully refactoring it was not realistic within the time frame.

Managers

We built a system that makes it easier to add new animations for weapons. We also made a Python-based Weapon Overlay Tool, which helped us add new weapon sprites to the player without manually editing every combination.

Code Quality and Maintainability

We used SonarQube throughout the project and spent the final iteration fixing issues related to cognitive complexity, loop structure and bugs such as integer overflow. JavaDocs were added for important classes and methods.

Overall, the code is reasonably easy to navigate, but the model package should be divided into more logical sub-packages.

Testing and Coverage

Package | Instruction Coverage | Branch Coverage

controller | 100% | 97%
 model.events | 100% | -
 model.weapon | 97% | 97%
 model.enemy | 97% | 87%
 model.collision | 93% | 79%
 model.levelsystem | 93% | 76%
 model.projectile | 92% | 63%
 model main | 91% | 79%
 app | 38% | 40%
 Total | 89% | 77%

We are above our branch coverage goal and well above the instruction coverage requirement. The lowest coverage is in the app package, mainly because it contains application setup and view-related code, which is harder to unit test. The sound package also has lower branch coverage because audio behavior is difficult to test automatically.

The tests gave us more confidence when refactoring AppModel and AppView. Overall, the main gameplay logic is well tested, while the remaining weak areas are mostly application setup, audio and view-related functionality. If we continued, we would make these parts more testable by separating more logic into dedicated classes.

Eksamensoppgaver

Her er en enkel transkripsjon av alle spørsmål og svar i eksamenen, basert på PDF-en du lastet opp.

1.1 Bakgrunn [5 poeng]

Spørsmål

Det kan være krevende å sette seg inn i et nytt prosjekt. Ofte må du lære mange verktøy, biblioteker og teknologier. I tillegg må du kanskje jobbe på nye eller uvante måter.

Tenk gjennom erfaringene dine, og nevnt to ting...

- a) ...som du føler du eller teamet ditt hadde hatt nytte av å kunne bedre før dere begynte på prosjektet,
- b). ...eller som du trodde ville være viktige på starten av semesteret, men som viste seg ikke å være så nødvendige.

Du kan velge to ting fra a eller b, eller gi ett eksempel fra hver. Husk å begrunne svaret.

(Ca. et avsnitt på hver ting)

Svar

En ting vi hadde hatt nytte av å kunne bedre før vi begynte var Git. Vi hadde en bratt læringskurve på branching og rebasing. Noen på teamet hadde god kontroll på dette fra egne prosjekter, men majoriteten hadde kun erfaringer fra INF101 og INF102 hvor man kun bruker add, commit og push til et repo du styrer alene. Vi løste dette tidlig med å ha en felles gjennomgang av git commands og andre relevante verktøy som lazygit. Vi lagde også retningslinjer på hvordan commit meldingene skulle være med forskjellige tags og branch navngivning.

Vi trodde på starten at det var viktig at alle hadde komplett og konstant innsikt på alt de andre gjorde, dette viste seg å ikke stemme da de ukentlige møtene (scrum-stil) ga alle kontroll på prosjektet som en helhet og lot oss fokusere på våre egne oppgaver mens Scrum-master passet på at alt gikk som det skulle. Vi endte også opp med et Trello-board som hjalp oss se hvor langt vi hadde kom i den gjeldene sprinten. Dette førte til at vi slapp å ha konstante meldinger med spørsmål til Scrum-master og gjorde oss mer effektive.

1.2 Teamarbeid [5 poeng]

Spørsmål

Tenk gjennom erfaringene dine fra vårens prosjektarbeid.

Hva synes du er de to viktigste tingene du har lært om teamarbeid i løpet av arbeidet med prosjektet? Forklar.

(Ca. et avsnitt på hver ting)

Svar

Det viktigste jeg lærte var hvordan Scrum virket i praksis. En agil arbeidsmetodikk med et team var noe jeg aldri hadde erfart før, men hadde hørt veldig mye om i jobbintervjuer. Det ble en veldig fin erfaring å se hvordan et team kan bli så samkjørte og alle etterhvert ble selv drevene når vi fulgte sprintene, standup og rollene man vi ble enige om. Dette får jeg bruk for i min gjeldene jobb nå!

Den andre tingen jeg lærte var viktig, var alt alle har forskjellige timeplaner og ikke alle kan ha like mye fokus og belastning hver uke. Å ha et møtereferat/log fra møtene mellom sprintene som gjør at man alltid kan gå tilbake å lese om man har glemt noe. Grunnen til dette ble viktig var at når det kom innleveringer i andre fag så ble det plutselig flere dager mellom møte og når man skulle jobbe med prosjektet. Vi på teamet går forskjellige linjer så vi hadde veldig varierende timeplaner. Da å kunne gå tilbake og lese gjennom møte

gjorde at man raskt kunne starte igjen med sine oppgaver. Det eliminerte mye unødvendig kommunikasjon og meldinger som oppsto i starten da møtereferatene ikke var gjennomført like nøye.

1.3 Fremtid [5 poeng]

Spørsmål

Nevn to ting du synes du bør lære mer om for å bli en god programvareutvikler, og forklar hvorfor de er viktige.

(Ca. et avsnitt på hver ting)

Svar

Den første viktige tingen er hvordan man fort kan sette seg inn i andres kode base med å lese diagrammer. Som deltids-ansatt som utvikler brukte jeg mye tid i starten av hvert prosjekt bytte, på å bare sette meg inn i hvor forskjellig logikk var, og hvordan prosjektet var strukturert. Å klare å lese klasse diagrammer, flyt modeller og fort absorbere annen dokumentasjon, gjør at man raskere kan starte å produsere kode.

Den andre viktige tingen er hvordan man skriver kode etter retningslinjer og regler satt av noen andre/arbeidsgiver. De fleste har sine måte å gi navn til klasser og metoder, hvordan modulere koden, pakke/mappe struktur. Men når man kommer i jobb og skal inn på et prosjekt hvor det jobber flere titalls andre så er det viktig at man følger generelle retningslinjer og vet hvilke deler/steder av koden som mest sannsynlig har retningslinjer man må følge. Dette gjør det mye lettere for resten av teamet og forstå koden du har skrevet.

2.1 Git arbeidsflyt [6 poeng]

Spørsmål

Se for deg at du og noen andre studenter skal begynne på et nytt programmeringsprosjekt (f.eks. som del av INF218/219). Dere er et lite team på 4–6 utviklere. Gruppen skal kode i Java. Du er valgt som teamets Git-ekspert

a) Hvordan ville du lagt opp Git arbeidsflyten slik at dere kan samarbeide best mulig, både med vedlikehold og videreutvikling? F.eks. med tanke på branch-struktur, navngiving og bruk av branches, om man skal pushe direkte til main eller bruke merge requests osv.

b) Et av de andre teammedlemmene kan ikke så mye om Git, og spør deg om hjelp med en merge konflikt. Forklar kort hvordan hen bør gå frem for å løse merge konflikten.

(2–3 avsnitt)

Svar

a) Jeg som Git-ekspert ville laget noen retningslinjer som alle på teamet måtte fulgt og som hadde gjort det ryddig og oversiklig for alle: vi hadde brukt tags som: feat, realease, hotfix, cleanup, refactor.

1. Vi har to Main branch: main som er en stabil og produksjonsklar branch og develop som er en integrasjons branch.

2. Ingen skal jobbe direkte på main.

3. Alt skal først pushes til deveop for integrasjons testing.

4. alle skal jobbe på en egen branch.

5. hver branch skal bare dekke en oppgave

6. branch skal navngis etter: ``<tag>/<navn>``

7. alle skal lage en merge request til develop først som må ble sett over av en annen utvikler.

b)

Merge konflikten skjer når det er gjort endringer (ofte noen andre har merget) på et sted du har endret/lagt inn/slettet kode fra når du gjorde din forrige rebase. For å gå fram for å løse en merge conflict bruker vi først bare git rebase ``<branch-navn>``. Om endringene har en større konflikt å bruker vi git rebase -i HEAD<n (head number/id)> for å få en interaktiv rebase. Da kan vi manuelt se over konflikt og velge hvilke versjon vi vil bruke. Når dette er gjort skal du kunne pushe og lage en PR. Siste løsning er å manuelt gå gjennom konflikten og endre din kode, for å så bruke git add . og commit.

2.2 IDE [4 poeng]

Spørsmål

Java-kode er vanlig tekst og kan skrives i en vanlig teksteditor. Likevel velger de fleste å bruke et integrert utviklingsmiljø (Integrated development environment – IDE), spesielt i større prosjekter. IDE-er har mange nyttige hjelpemidler, for eksempel autofullføring, navigasjon, dokumentasjonsvisning og mye annet. De gjør det også enklere å holde orden på prosjektet.

Med støtte fra en IDE blir det enklere å skrive korrekt kode, finne frem i ukjent kode og gjøre endringer eller vedlikehold. Eksempler på vanlige IDE-er for Java er Eclipse, IntelliJ IDEA, NetBeans og VSCode.

- a. Hvilken IDE (eller editor) brukte du vanligvis til å redigere koden i semesterprosjektet? Brukte alle på teamet samme IDE?
- b. Hva tenker du er de nyttigste tingene å se etter når du skal velge IDE, slik at du får best mulig hjelp og støtte i utviklingsarbeidet?

(1–2 avsnitt)

Svar

a) Vi på hele teamet brukte IntelliJ som hovededitor. Jeg brukte også neovim når jeg skulle skrive masse "boilerplate" kode på en gang, uten å trenge å kjøre programmet. At hele teamet brukte IntelliJ gjorde formatting veldig lett, da vi alle kunne de samme plugins og instillinger.

b) Det nyttigste for meg er hvor mye IDE kan gjøre av oppsettet for deg, og maven build i dette prosjektet. At den har gode plugins, vim support, formatering, LSP-support for språket. At den er lett å navigere uten å manuelt klikke gjennom mappene. Det er også viktig at den har mange analyse-verktøy for testing og effektivitet. Dette gjør det lett å se hvor forbedringspotensialet ligger.

2.3 CI/CD [3 poeng]

Spørsmål

Under vårens prosjektarbeid var GitLab satt opp med kontinuerlig integrasjonstesting (CI/CD – Continuous Integration / Deployment – vi brukte bare CI), slik at hver gang du pushet endringer til git.app.uib.no ble det startet en såkalt pipeline på serveren, som sjekket ut den nyeste koden, kompilerte den og kjørte JUnit-tester og evt. andre tester. Resultatet av testkjøringen ble deretter gjort tilgjengelig på prosjektsiden.

a. Når du utvikler kjører du vanligvis testene selv på din egen maskin før du committer og pusher. Hvorfor kan det likevel være nyttig å også ha kontinuerlig integrasjonstesting når du pusher til serveren? Forklar.

b. Opplevde du i løpet av semesteret at CI-serveren fant feil / problemer du ikke var klar over før du pushet?

(1–2 avsnitt)

Svar

a) Det er ikke alltid din pc/mac har samme konfigurasjon og env som serveren. At alle må kjøre koden på serveren gjør at man er sikker på at koden du pushet kan kjøre uten feil på serveren. Pipeline feil gjør også at andre på teamet kan se hva som ble feil, det gjør det også lett å "roll-backe" koden om en feil skulle oppstå. Det gjør også at man ser feil med fil-stier til ressurser om man har skrevet det feil, da serveren ikke har samme mappe struktur som deg.

b) Nei, den fant aldri uventede feil på mine push. Vi hadde en på gruppen som hadde komplett filsti til en lydfil i koden sin og ikke sti fra src folder. Så der fanget pipeline en feil som ikke ble sett at utvikleren.

2.4 Statisk analyse [2 poeng]

Spørsmål

I løpet av semesteret satte vi også opp noen statiske analyseverktøy – SpotBugs og SonarQube.

Slike verktøy analyserer koden – dvs. teksten og de kompilerte klassefilene – uten å kjøre den. Målet er å finne feil og potensielle problemer – samt ting som ikke egentlig er feil men kan være dårlig praksis eller føre til vedlikeholdsproblemer. SonarQube presenterer funnene på en nettside, sammen med litt statistikk (blant annet test coverage).

Eksempler på ting SpotBugs og SonarQube sjekker etter er: metoder som er for kompliserte, literaler som burde defineres som konstanter, feltnavn som kan misforstås, definisjoner som burde vært private, protected eller final osv. Java-kompilatoren gjør også litt statisk analyse – blant sjekker den for typefeil, variabler som ikke blir brukt osv.

a. Tittet du på SonarQube-siden i løpet av semesteret? I så fall, var noen av tilbakemeldingene nyttige?

(SonarQube-oppsettet var litt eksperimentelt, så det er helt rimelig om du ikke sjekket det.)

Du la antakelig mye arbeid i å skrive JUnit-tester i løpet av semesteret. Men trenger vi egentlig å skrive enhetstester når vi har fancy analyseverktøy som SonarQube – eller trenger vi SonarQube når vi har tester?

b. Gi et eksempel på feil/problemer som statistisk analyse kan oppdage, men som er vanskelig å oppdage med enhetstester.

c. Gi et eksempel på feil/problemer som er greit å oppdage med tester, men som vil være vanskelig for et statistisk analyseverktøy å oppdage.

(Noen setninger om hvert punkt.)

Svar

a) Jeg var inne å så på sonarqube for å rydde opp i koden før innlevering, den viste meg alt av små "errors" som ubrukte imports, typos og andre små forbedringer som ga oss høyere kodekvalitet.

b)

- * Ubrukte imports

- * Typos

- * kodeduplikat

- * innkapsulering, at man marker det som ikke blir endret med final, osv.

c) statistisk analyse kan ikke oppdage:

- * kjøre feil

- * logikk feil, at funksjonen gjør det den faktisk skal

- * sideeffekter, at funksjonen ikke endrer på ting den ikke skal endre.

3.1 Valg av abstraksjoner [5 poeng]

Spørsmål

En viktig designavgjørelse er hvilke ting som skal være egne abstraksjoner (egne interfaces), og hva som kan representeres direkte med primitive typer og klasser fra Java-biblioteket eller andre vanlige biblioteker. Førstnevnte kan gi et API som er bedre tilpasset bruken, mens sistnevnte har fordelen at folk gjerne kjenner det fra før (og det blir mindre jobb for deg). For eksempel:

Vi kan lage en egen type Map for kartet, eller vi kan bruke Javas MapSquare[][] eller en generell grid-datastruktur Grid<MapSquare>.

Posisjoner kan enkelt representeres som tall, x og y – men kanskje det er lurt å ha metoder for å regne ut retninger og avstander? Burde vi i så fall lage en egen Position, eller heller bruke f.eks. LibGDX sin Vector2?

Retninger kan være tall (i grader) eller vektorer – men det er også mulig det er nyttig med en egen Direction abstraksjon

Hvilke designvalg ville du gjort for kart, posisjon og retning? Husk å begrunne svaret.

(1–3 avsnitt)

Svar

Jeg tenker siden det er biologer som skal bruke API, som mest sannsynlig ikke har mye koderfering, er det best å lage en egen type Map for kartet, jeg vil også ha en MapCell record som holder (Position, T thing) så posisjon og en ting som er på kartet. Vi brude også bruke en egen Position, med felt variabler som gir mening for biologer som koordinater til geiten. Vector2 klassen kan bli brukt innad i Position til å regne ut koordinater hvis vi vet vilke koordinater som Map spanner. Det viktigste er at når biolognen skal ta over så har de mulighet til å forstå hva ting betyr og gjør. Retningen bør også ha abstraksjon i form av et Direction Interface. Dette kan ha metoder som getDirection (som kan retunere circa retning i form av nord, sør, øst, vest) og en metode som

getAngleDircetion som gir nøyaktig retning i grader. Totalt sett blir dette lettere for en biolog å forstå da de slipper å lese seg opp på hele Vector2, MapSquare, osv og kun blir presentert med relevante og selvforklarende metoder.

3.2 Å mutere eller ikke mutere? [2 poeng]

Spørsmål

Et annet valg er om vi skal kunne oppdatere objektene (de er muterbare, mutable), eller om hver utregning gir et nytt objekt (de er ikke-muterbare, immutable, og kan ikke endres etter at de er opprettet). For eksempel er LibGDX sin Vector2 mutable, mens Java sin String er immutable.

Hvorfor kan det være nyttig å bruke ikke-muterbare objekter? Forklar kort.

Hva ville du valgt for dine abstraksjoner? Forklar kort.

(Ca. 1 avsnitt)

Svar

Om et objekt er ikke-muterbart så kan man unngå mange feil og bugs. At man eksplisitt på reassign et objekt for å endre på det gjør koden mer leselig og forståelig. Dette er spesielt relevant om man skal gi en verdi til en funksjon som da endrer på objektet når dette ikke var tanken. Med mine abstraksjoner ville jeg holdt objektene ikke-muterbare så biologene blir tvunget til å eksplisitt spesifisere at de vil endre med å reassign. Det jeg da må passe på et at metode-navnene må vise dette. Så istedetfor position.move(delta), så ville jeg brukt position.movedBy(delta) for å gjøre det mer forståelig.

3.3 Java interface [6 poeng]

Spørsmål

Skriv Java-kode (kun interface) for eventuelle abstraksjoner (kart, posisjon og retning) du valgte å ha med i designet i de første deloppgavene.

Ta med metoder du mener er nødvendige, og metoder du tenker vil være nyttige for de som implementerer geite-oppførselen.

Skriv kort JavaDoc-dokumentasjon – en linje eller to om hva metodene gjør. Du trenger ikke ha med full @param beskrivelse av parameterne, men ta med eventuell informasjon brukeren må være oppmerksom på – spesielle krav til parametre, at en metode kan returnere null, om metoder må kalles i en spesiell rekkefølge osv.

Svar

```
public record mapCell(Position pos, T thing)

public enum Dir {
    north,
    south,
    east,
    west
}

public record Step(int x)

public interface Map {
    //T is an enum representing possible "things" that can be
    placed on the map

    //return a list of what is on that mapSquire,
    mapCell get(Position pos)

    //changes that mapCell to hold t
    set(T t, Position pos)

    //returns all mapCells
    mapCell[] getAll()
}

public interface Position {
```

```

//returns a new position moved to pos (nice if Position
holds other values)
Position posMovedTo(Position pos)

//returns a new position shifted by x animalsteps in
direction of animal
Position posShiftedBy(Step x)

}

public interface Direction {
//returns direction in terms of north, west, south, east
Dir getDirection()

//returns degree of direction
Degree getDegree()

}

```

3.4 Gammel kode [3 poeng]

****Spørsmål****

En annen gruppe studenter laget en prototype av geitesimuleringen i fjor. De hadde ikke tatt INF112, og kunne betydelig mindre om programutvikling enn deg. I kodeeksempelet (se vedlagt PDF) kan du se deres implementasjon av `Goat::findFood()`, som skal lete etter mat i nærheten og snu geiten i riktig retning om den finner mat. Studer koden – du trenger ikke bruke tid på alle detaljene i hvordan den virker, det holder å forstå hva den gjør.

Tenk gjennom designprinsippene (SOLID, osv.) og det du har lært om kodekvalitet og vedlikeholdbarhet. Hva tenker du om implementasjonen av `findFood()`? Er dette et bra grunnlag for vedlikehold og fremtidige utvidelser?

(Ca. 1 avsnitt)

****Svar****

Koden burde vært mye mer delt opp og brukt hjelpemetoder som er mer abstrakte. Det bryter direkte med single-responsibility prinsippet da findFood metoden gjør. The open-closed prinsippet er kanskje ikke direkte brukt med koden er så inviklet med mange nested for loops at den blir lukket for utvidelser. Det er ingen brudd op the Lisknov substitution prinsippet som jeg finner. Det er heller ikke brudd på the interface segregation prinsippet da det ikke er brukt noe interface, dette er også kritikk da interface hadde gjort koden mye bedre. Det er stort brudd på The dependency inversion prinsippet som sier at "depend on abstractions, not concretions". Her er det nesten ingen abstraksjon og alt baseres på primitive verdier.

3.5 Renovasjon [4 poeng]

****Spørsmål****

Anta at noen allerede har implementert og testet API-et du laget, og at Goat-klassen er oppdatert til å bruke det nye API-et i stedet for int[][] foodMap o.l.

Lag en ny implementasjon av findFood() som bruker API-et ditt. Du står fritt til å endre argumenter og gjøre passende antakelser om feltvariabler og andre metoder i Goat.

****Svar****

```
```java
```

```
public interface animal {
```

```
//since we are adding more animals later i collect all shared methods in an
```

```
//returns the steps to a particular mapCell
```

```
Step stepsFrom(mapCell);
```

```
//returns the direction toward a particular mapCell
```

```
Direction directionTo(mapCell);
```

```
}
```

```
public class Goat implements animal {
```

```
 Position pos;
```

```
 Direction dir;
```

```
 Position dest;
```

```
 public boolean findFood(Map map, Step maxSteps){
```

```
 MapCell cell = map.get(cell);
```

```
 if(cell.T instanceof Food){
```

```
 return true
```

```
 }
```

```
 for(MapCell cell : map.getAll()){
```

```
 if(cell.T instanceof food) && stepsFrom(cell) < maxSteps{
```

```
 dest = cell;
```

```
 direction = directionTo(cell)
```

```
 return true;
```

```
 }
```

```
 }
```

```
 return false;
```

```
 }
```

```
@Override
```

```
public Step stepsFrom(MapCell){
```

```
 /*...*/
```

```
}
```

```
@Override
```

```
private Direction directionTo(cell){
```

```
 /*...*/
```

```
}
```

```
 ...
```

```

```

### ## 3.6 Refleksjon [3 poeng]

**\*\*Spørsmål\*\***

Hva tenker du om din implementasjon av findFood()? Er den en forbedring i forhold til den gamle?

(Noen få setninger)

**\*\*Svar\*\***

Min er kortere og bruker mer abstrakte løsninger som gjør den leselig og lett å forstå. Bruken av metoder som er selvforklarende gjør også den lett å justere på. Logikken er også mye lettere å følge: Først sjekk om det er mat der geiten står, hvis ikke se gjennom alle cellene til kartet, om cellen har mat og er innenfor rekkevidde. Gå ditt!

---

#### ## 4.1 Brukerhistorier [3 poeng]

##### **\*\*Spørsmål\*\***

Som UiB-student har du erfaring som bruker (i student-rollen) i den digitale eksamensløsningen Inspira. Tenk gjennom erfaringene dine, brukeropplevelsen og hva som er viktigst for deg som student når du bruker Inspira til å besvare eksamen.

Med utgangspunkt i student-rollen og deg selv som persona, skriv to brukerhistorier for en digital eksamensløsning. For eksempel noe du synes fungerer spesielt godt eller dårlig, eller noe du savner i dagens løsning.

(To punkter)

##### **\*\*Svar\*\***

Som en elev vil jeg ha kodeautofylling og LSP-support slik at man har mer tid å bruke på kvalitet og ikke skrive mye repetitivt.

Som en elev vil jeg ha mulighet til å koble til eget trådløst ergonomisk tastatur for å slippe skrive pauser grunnet smerter i håndledd.

---

#### ## 4.2 Universell utforming [4 poeng]

##### **\*\*Spørsmål\*\***

Universell utforming av programvare er viktig for at så mange som mulig skal kunne delta i samfunnet på en likeverdig måte, uavhengig av eventuell funksjonsnedsettelse, kulturell bakgrunn, kjønnsidentitet, alder osv. Både EU, USA, Norge og andre land har regler om dette – i Norge er det regulert gjennom Likestillingsloven og Forskrift om universell utforming av IKT-løsninger, og UU-tilsynet fører tilsyn med og gir råd om universell utforming.

a. Hva slags konsekvenser kan det eventuelt få for en virksomhet (privat eller offentlig) hvis de tilbyr en nett-tjeneste som ikke oppfyller minimumskravene til universell utforming?

(Svar kort.)

b. Universell utforming av eksamen er viktig for å sikre like muligheter i samfunnet – det er noe både utviklere og forelesere må være oppmerksom på. Tenk gjennom brukeropplevelsen på eksamen i dag og eventuelt andre eksamener du har tatt.

Kommer du på et tilfelle der du føler at brukeropplevelsen kunne være til hinder for enkelte studenter, selv om den kanskje er grei for de fleste? Forklar kort.

(Det trenger ikke være snakk om et faktisk brudd på regler eller retningslinjer, bare at du tenker det kan være til hinder for noen. Hvis du ikke kommer på noe eksamensrelatert, kan du eventuelt tenke på andre tjenester/applikasjoner du bruker.)

(1–2 avsnitt)

**\*\*Svar\*\***

a) Konsekvenser kan være alt fra tapte kunder som påvirker virksomheter, til straff fra myndighetene i form av bøter. Siden det også er veldig mye fokus i samfunnet på likestilling så kan virksomheter også oppleve press/boikotter fra samfunnet om de ikke følger universell utforming.

b)

Jeg tenker at det kan være hinder for folk fra en annen kulturell bakgrunn at eksamen digitalt krever at man klarer å skrive relativt fort på pc, da ofte ord-kravene er høyere enn når man skriver for hånd. Elever fra spesielt familier fra andre kulturer som ikke nødvendigvis er oppvokst foran tastaturet og kan skrive ute å tenke seg om bruker mye tid og krefter bare på å beherske skrivingen.

---

## 5 Poeng fra semesterprosjektet [40 poeng]

**\*\*Spørsmål\*\***

God sommer!

**\*\*Svar\*\***

Ingen tekstlig besvarelse.